FIRLT : Fault Injection in Rust with Lazart Tool

Timothe Baleras
Ensimag,UGA,Verimag
timothe.baleras@grenoble-inp.org

Supervised by: Laurent Mounier,Etienne Boespflug

1 Abstract

The Rust programming language [1] is rapidly being adopted in safety- and security-critical software. Its compile-time guarantees, thanks to borrow checking and some other language constructs — exhaustive pattern matching, mandatory error handling through `Option` and `Result`, and automatic bounds check — eliminate entire classes of memory-safety vulnerabilities that have plagued C and C++ for decades.

Fault injection is a critical security concern that can cause hardware to malfunction or be exploited by attackers. Rust is not protected from fault injection, as most of its protections are made at compile time and can be faulted during runtime. That is why tools like Lazart are essential for identifying faults and evaluating code—including C and Rust—to protect systems from such vulnerabilities.

In this study, we extend the tool Lazart [2] to support the Rust language by updating its LLVM backend in order to support the pipeline of `cargo` and `rustc`, and by adding a new attack model giving Lazart the ability to change the address of a pointer during execution. A new option has been added to the Lazart tool giving the ability to generate a control flow graph of the compiled code or its mutated version. Finally, we ported the FISSC fault injection collection from C to Rust and provided a more Rust-idiomatic version of the collection. The results section covers the findings of FISSC and evaluates the impact of the new control flow graph generator. It highlights that Rust and C diverge both in design and in their approach to control flow by utilizing instructions typically reserved for pattern matching.

2 Introduction

2.1 Fault injection

In computer science, many types of faults can cause a system to behave unexpectedly. Faults may occur during the development or execution of a program and can originate from the software, the hardware, or both. Faults come in a wide variety : they can be syntax errors that cause the script to fail to compile, generated warnings, or crashes with an error during compilation. Faults can be caused by human error, misconfigurations that were not anticipated, or malicious individuals exploiting security vulnerabilities. One important type of fault is **fault injection**. Fault injection has been around for a long time, and we found traces of papers dating from the 1970s, but they were never published ; the oldest one published is from 1995 [3]. This fault can be caused by various techniques, including hardware, such as clock glitches [4, 5], voltage glitches [6, 7, 8], laser [9, 10], or electromagnetic faults [11, 12], as well as software, such as Rowhammer [13]. It can occur accidentally, for example due to cosmic rays, or deliberately by a mischievous individual ; in the latter case, we talk about a fault injection attack. To deal with fault injection, developers use several countermeasures [14, 15, 16] or tools [17, 18, 3, 19] to protect their programs. One such tool is called Lazart[2]. It is a multi-fault injection tool that mutates the code at the LLVM level in order to simulate the effects of fault injection attacks.

2.2 Rust

The Rust language is a recent language defined as a successor to C and C++ ; while C/C++ have a lot of memory access bugs such as buffer overflows or data race issues.

Rust has stronger protection over memory access and concurrency over variables ; variables are tracked with an element called the borrow checker that tracks the lifespan and ownership, as well as their access. While the borrow checker protects against bad access of data in the software program.

The same cannot be said for hardware failure, which may cause some protections to be countered. An

example of such failure is given in subsection 4.2 of section 4. Subsection 4.2 is only a summary of the full version discussed in the FDTC paper (under submission) [20].

We can take the buffer overflows of a table rust protection as an example. `rustc` will add a test on the code at compile time, to identify if you reading in or out the table, and will launch a pannic if it's outside the range of the table. injected a faute at the moment of the condition can allow the program to read outside the table, and thus counter the protection set by Rust language itself.

An other example, is the change pointer fault. It is explained in the subsection 4.2.

2.3 Contribution

The purpose of this report is to show how Rust differs from C and C++ regarding offensive fault injection. To answer that, there are two different parts :

— **Extend Lazart to support Rust** : discussed in section 4, this shows the instructions that have to be added in order to analyze Rust programs without losing attacks. In addition, we also discuss the change pointer fault in this section.

— **Porting the FISSC benchmark to Rust** : this point discusses FISSC (Fault Injection and Simulation Secure Collection) [14]. We analyze two programs of FISSC in Rust and C and compare them to each other. See section 6

The report is organized as follows. Section 3 presents the Rust programming language, related work concerning Rust and fault injection, and the tool Lazart. Section 4 describes the extension of Lazart to support the Rust language and the addition of a new attack model for C and Rust languages. Section 5 presents a new option of the Lazart tool giving the possibility to generate a control flow graph (CFG). Section 6 shows the comparison made on the FISSC benchmark and a Rust implementation designed to match the C implementation's attack surface. Section 7 shows results for the FISSC comparison of a Rust implementation, and findings that were produced from control flow graph analysis. And finally, Section 8 summarizes and proposes some perspectives to this work.

3 Related work

In this section, we talk about related work and a description of the Lazart tool.

3.1 Fault injection

Fault injection is an old, well-studied attack model in security and cryptology that can be created at the hardware or software level [15]. While multiple methods exist to simulate fault injection, one approach involves hardware-based techniques, such as [16, 18]. Another method to simulate fault injection is using software tools [2, 21]. One such software tool is called Lazart [2]. A countermeasure benchmark, developed using the Lazart tool by several researchers, is FISSC [14].

While a few studies have been conducted on countermeasures in Rust [22, 23], there is little research that has been conducted on comparing the results between C and Rust for countermeasures.

Furthermore, little research has been done on the effectiveness of the protection of the countermeasure from compiler optimizations. We addressed this gap by highlighting the work of Sébastien Michelland [24].

3.2 Lazart tool

Lazart [2] is a high-level fault injection tool. It has been developed by Verimag since 2015. It allows for identifying vulnerable paths in code and can be used to evaluate the accuracy of countermeasures against a given fault model and a security property. Unlike other similar tools, Lazart is one of the few that are capable of performing multi-fault attacks. Lazart is able to analyze C or C++ code by compiling it using the Clang compiler. It uses one of the features of Clang to generate LLVM bytecode (which is intermediate code).
LLVM is a middle component of the compiler and is designed for compile-time, link-time, and run-time optimization. The LLVM framework is run by Clang after the frontend pipeline is completed. Lazart is composed of five steps (A Lazart arckitecture is provided in figure 1) :

1. Preprocessing and compilation of the input code.
2. Mutation of the code with Wolverine.
3. Dynamic-Symbolic Execution with KLEE [25, 26].
4. Trace parsing from KLEE's outputs.
5. Analysis.

Lazart can support 4 fault models :

— Test inversion (modification of the evaluation of a branch condition) ;
— Data mutation (modification of the loaded values) ;
— Instruction skip ;
— Call another function (function switch) ;

3.3 Rust Language

Rust is a general-purpose programming language which emphasizes performance, type safety, concurrency, and memory safety. Rust had its first version released in 2006. One of the features of Rust is the way how it handles the memory safety and concurrency of a program. Unlike other languages which use a garbage collector, Rust uses a "borrow checker" that tracks the lifespan of a variable as its owner. And deals with data races.

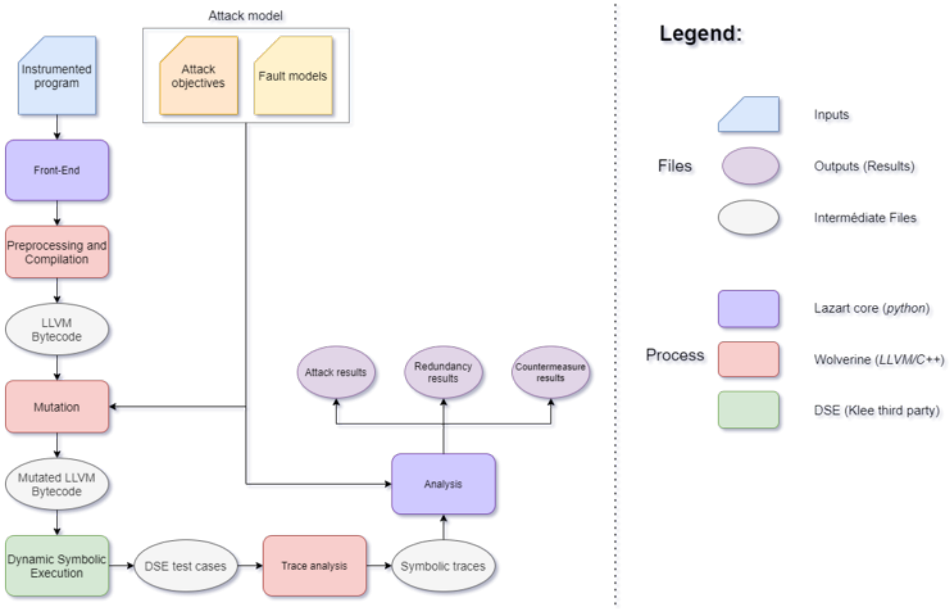

FIGURE 1 – Lazart tool architecture

Why use Rust ?

We give several reasons for using Rust :

— Rust uses the same LLVM as clang does in C/C++. So it makes Rust compatible with Lazart.

— Rust is a recent, still updated programming language.

— Rust is used for embedded systems.

The usage of no_std

One point that needs to be explained in this report is the flag no_std. Using the no_std attribute stops the compiler from linking std into the crate and only links core. This method reduces binary size by omitting Rust standard library components, thereby facilitating Klee's execution process. It limits :

— Heap allocation.

— File I/O, threading, and networking.

— Many common traits and types (e.g., std : :io : :Error).

4 RUST ported in lazart

Some modification have been made in order to support the Rust language.

4.1 Limitation of Lazart for Rust

Although Lazart works on LLVM and rustc is capable of producing LLVM, Some limitation was encountered, when trying to analyse Rust programmers with Lazart.

LLVM version. The LLVM version of Lazart was updated from LLVM 9 to LLVM 14, as the Rust tool chain matching LLVM 9 is too old for cargo to fetch crates from registry.

Test inversion extension. rustc makes an extensive use of LLVM's `select` and `switch` instructions compared to C compilers (here `clang`). Rust is capable to generate `switch` and `select` from simple branching (example : condition branch), even if the `switch` instructions is often used for pattern matching. The test inversion model has been extended to support `switch` and `select` LLVM instructions in order to be able to fault some control flow constructs (to match C example in FISSC) or some language constructs (e.g. pattern matching).

4.2 The Change Pointer fault model extension

A new form of attack as been added. Since Rust borrow checker guarantees safety at compiled time. Nothing guarantees those safety to protected at runtime are they are purely static. Since developers rely on the borrow checker for their program and to enable optimisation. One way to break the borrow checker guarantees is to consider a fault model of *change pointer* making possible to redirect a mutable reference to another. This fault is made possible with the help of the `unsafe` keyword. At lower representation levels, the fault can map a skip on an address computation, or a fault on a register or the stack.

Change Pointer fault model The Change Pointer (CP) fault model, implemented in Lazart for Rust and C, redirects a reference so that it point to another object. Fault yielding a dangling or wild pointer are out of scope due to them raising a trap and become easily detectable. A redirection between two legitimate references is silent and is exactly what defeats a borrow checker invariant.

Example : privilege escalation by reference swap. Listing 1 shows a typical use-case example of the model in Lazart.The target program uses the function `connect` to legitimately set the session with a specified duration.The two *Change Pointer* injection points (using the macro `_LZ__change_ptr_R!(good, bad, id, τ)`) modify the nominal parameters `connected` and `duration` to `admin` and `sold` respectively.

Listing 1 – Privilege escalation via change-pointer faults.

```
fn connect(session: &mut bool, amount: &mut i32) {
    *session = true;   // legitimate, non-privileged write
    *amount  = 100;
}

fn main() {
    let (mut admin, mut connected) = (false, false);
    let (mut sold,  mut duration)  = (0, 0);

    // Pointer change on function call.
    connect(_LZ__change_ptr_R!(&mut connected, &mut admin,
    ↪ "is_admin",  bool),
            _LZ__change_ptr_R!(&mut duration,  &mut sold,
    ↪ "have_sold", i32));
    _LZ__ORACLE!(admin & (sold == 100)); // always false
    ↪ without faults.
}
```

Lazart finds an attack combining a fault on each injection point to force a mutable aliasing that is forbidden by Rust, leading to privilege escalation. Although expected, these results show that the memory safety *by construction* does not imply *fault injection* resistance.

Note : the implementation of such faults is not present in this paper (see Disclaimer at the end) and is discussed in the FDTC paper (under submission) [20].

5 CFG

In this section, we talk of a new optional feature of Lazart. The possibility to generate a control flow graph from the normal or mutated version.

5.1 Control flow graph

Reminder : A Control Flow Graph is a representation, using graph notation, of all paths that might be traversed through a function during its execution, or control flow. We generated a control flow graph from the generated LLVM code, which is stored in a ".ll" file.

With Lazart, it is also possible to generate a control flow graph from the mutated code produced from Wolverine or from the program compiled. We used the control-flow graphs to identify and compare the instructions used for each language (C and Rust) in order to understand their differences.

See the example in Appendix A in Figure 6.

The control flow graph was primarily used to understand the optimization of `rustc` with their own boolean and give an additional explanation of one of the findings with Table 1 : `verify_pin` (see below in section 6) in the section 7.

6 Rewritting FISSC in Rust

FISSC [1] (Fault Injection and Simulation Secure Collection) is a public code collection dedicated to the analysis of code robustness against fault injection attacks. It contains two pieces of code that we will discuss in this section :

— The first code evaluated is `verify_pin`, a program used to validate a PIN code on a PIN pad.

— The second code is `memcmps`, a secure version of `memcmp`.

Originally written in C, both code segments were ported to Rust, following a C-style programming approach, and were subsequently examined using Lazart. Then, each code in both languages was analyzed and compared. In addition to the comparison, we have also written a more idiomatic version of the FISSC collection.

6.1 *verifypin* collection

```
#include "verify_pin.h"
#include "lazart.h"

BOOL compare(uint8_t* a1, uint8_t* a2, size_t size)
{
    int i;
    _LZ__RENAME_BB("for_in_compare");
    for (i = 0; i < size; i++) {
        _LZ__RENAME_BB("check in compare");
        if (a1[i] != a2[i]) {
            return FALSE;
        }
    }
    return TRUE;
}

BOOL verify_pin(uint8_t* user_pin)
{
    _LZ__RENAME_BB("Try counter < 0");
    if (try_counter > 0) {

        _LZ__RENAME_BB("precompare");
        if (compare(user_pin, card_pin, PIN_SIZE)) {
            try_counter = TRY_COUNT;
            return TRUE;
        } else {
            _LZ__RENAME_BB("precompare_else");
            try_counter--;
            return FALSE;
        }
    }
    return FALSE;
}
```

FIGURE 2 – Verification of PIN code in C

The second example code implemented in Rust is **verifypin**.
The `verify_pin` program implements the PIN verification process of a smart card.
The source code in C shown in Figure 2 corresponds to a simple implementation without countermeasures other than a hardened boolean.

The **verifypin** function consists of two parts :

1. FISSC hyperlink : https://hal.science/hal-03784107v1/file/article-fissc.pdf

— Checking if there are enough tries remaining (line 20 in Figure 2).

— Resetting the try counter and returning `TRUE` if the `compare` function returns `TRUE`; otherwise, decrementing the number of tries remaining by 1 and returning `FALSE`.

`TRUE` contains the value `0xAA`.
`FALSE` contains the value `0x55`.

We consider `TRUE` and `FALSE` as hardened boolean values because they are encoded not just as single 0 or 1 bits, but as 8-bit patterns of 0s and 1s. Specifically, the pattern '01' represents `FALSE`, while '10' represents `TRUE`.
This approach makes it highly unlikely for a `TRUE` value to be accidentally flipped to `FALSE`, or vice versa.

The function takes one user PIN and returns either `TRUE` or `FALSE`.

The function **compare**, shown in Figure 2, compares each digit of the user input PIN with the one stored on the card. It returns `TRUE` if both PIN codes are identical and `FALSE` otherwise.

We studied two attack objectives with the help of an oracle :

— Returning `TRUE` when both PIN codes are different.

— Increasing the number of tries remaining in the case where both PIN codes are different.

The attack model used is the **test inversion**.

We implemented the Rust version of the source code shown in Figure 2, following the style used by C programmers (see Figure 4).

6.2 *memcmps* collection

The first example code implemented in Rust is `memcmps`.
`memcmps` is a secure version of `memcmp` that is available in C.

`memcmps` takes two input arrays that are initialized before the function is called.
`memcmps` returned two possible values :

— `TRUE_VAL`, corresponding to `0x1234`, when both arrays are equal.

— `FALSE_VAL`, corresponding to `0x5678`, when they are not equal.

In Figure 3, we show one of the three versions of `memcmps` that we implemented. This version used four function calls of `memcmp` as shown in lines 7, 9, 11, and 13 and two masks, `MASK` and `MASK2`, to prevent fault injection from affecting the returned value of the function.

— `MASK` corresponding to the value `0xABCD`

```
1  #include "memcmps.h"
2
3  int memcmp_stub(const char* str1, const char* str2, size_t count)
4  {
5      register const unsigned char* s1 = (const unsigned char*)str1;
6      register const unsigned char* s2 = (const unsigned char*)str2;
7      while (count-- > 0) {
8          if (*s1++ != *s2++)
9              return s1[-1] < s2[-1] ? -1 : 1;
10     }
11     return 0;
12 }
13
14 int memcmps(char* a, char* b, int len)
15 {
16     int result = FALSE_VAL;
17
18     if (!memcmp_stub(a, b, len)) {
19         result ^= FALSE_VAL ^ MASK; // result = MASK
20         if (!memcmp_stub(a, b, len)) {
21             result ^= MASK2; // result = MASK ^ MASK2
22             if (!memcmp_stub(a, b, len)) {
23                 result ^= TRUE_VAL ^ MASK; // result = MASK2 ^ TRUE_VAL
24                 if (!memcmp_stub(a, b, len))
25                     result ^= MASK2; // result = TRUE_VAL
26             }
27         }
28     }
29
30     return result;
31 }
```

FIGURE 3 – Securised version of memcmp in C

— `MASK2` corresponding to the value `0xF4F4`

`memcmps` works by using `MASK` on a variable called result (see line 5 on Figure 3).
If no fault is injected into the system, `memcmps` should return either true or false every time it is called.
So if both data blocks are the same, then the result will take the value of `MASK` after the first call at line 8.
the value take `MASK` $\wedge$ `MASK2` after the second call line 10.
the value take `MASK2` $\wedge$ `TRUE_VAL` after the third call line 12.
And finally the value takes `TRUE_VAL` after the fourth call line 14.

We studied two attack objectives with the help of an oracle :

— returning `TRUE_VAL` when both data blocks are **different**.

— returning anything but `FALSE_VAL` when both data blocks are equal.

The attack model used is the inversion test and data mutation, in the figure 3, one inversion test may cause the value to be neither `TRUE_VAL` nor `FALSE_VAL` (if the attack is done on the condition test line 7). Four inversion tests are needed to be able to return `TRUE_VAL` when the data blocks are different.

We implemented the Rust version of the code that would be written by C programmers.

Figure 5 shows the `memcmps` source code equivalent of Figure 3 in the Rust language.
In the Rust version of `memcmps`, we needed to change

```rust
use crate::common::*;
use crate::verify_pin::hardened_bool::{FALSE, TRUE};
use crate::*;

#[no_mangle]
#[inline(never)]
pub fn compare(a1: &[u8; PIN_SIZE], a2: &[u8; PIN_SIZE], size: usize) -> BOOL {
    _LZ_RENAME_BB_R!("for_in_compare");
    for i in 0..size {
        _LZ_RENAME_BB_R!("check in compare");
        if a1[i] != a2[i] {
            // IP0 / IP1 / IP2 / IP3 -> loop unrolling from the rustc compiler
            return do_real_load!(FALSE);
        }
    }
    return do_real_load!(TRUE);
}

#[no_mangle]
#[inline(never)]
pub fn verify_pin(
    user_pin: &[u8; PIN_SIZE],
    card_pin: &[u8; PIN_SIZE],
    try_counter: &mut u8,
) -> BOOL {
    _LZ_RENAME_BB_R!("Try counter < 0");
    if *try_counter > 0 {
        // IP4

        _LZ_RENAME_BB_R!("precompare");
        if do_real_load!(compare(user_pin, card_pin, PIN_SIZE).eq(&TRUE)) {
            // IP5
            // Authentication
            _LZ_RENAME_BB_R!("then");
            *try_counter = TRY_COUNT as u8;
            return TRUE;
        } else {
            _LZ_RENAME_BB_R!("precompare_else");
            *try_counter = *try_counter - 1;
            return FALSE;
        }
    }
    FALSE
}
```

FIGURE 4 – Verification of PIN code in Rust

the return value of `memcmp`, due to C not having a native boolean type.
`memcmp` returns *false* if both binary data blocks are the same and *true* otherwise.
We also used two flags for compiling Rust :

— The first is the use of the `no_mangle` flag on line 17 in Figure 5.
It is used to prevent Rust from renaming some of the blocks generated by the LLVM and to ensure the correct operation of Lazart.

— The second is the use of `inline(never)` on line 18, which forces Rust not to replace the function call with its body.
This is a type of optimization used by compilers to avoid context changes.

In memcmps programs, the data load(DL) faults are directly instrumented in the program (using `mut_dl_sym!` macro), instead of being specified by variable name. That prevent the propagation of the `len` value in SSA, which the DL model applies. This also prevents the constant folding for result that is reduced to a selection of constants depending on the execution path. Thus, direct calls to the DL mutation macro are made on each DL injection point.

6.3 A Rust version FISSC

we developed an idiomatic version of FISSC in Rust. The idiomatic version contain both previous examples, `verify_pin` and `memcmps`. Unlike the two previous examples, the goal is not to match one-to-one the C implementation but to provide a collection of code that can be reused with other formal-methods or simulation tools, or on another representation level. In addition to the idiomatic Rust versions of `verify_pin` and `memcmps` collections, we ported the remaining programs of FISSC to Rust. We briefly describe here the CRT-RSA, AES and GetChallenge programs from FISSC.

The `CRT-RSA collection` implements three versions of RSA signing using the Chinese Remainder Theorem (CRT). The program is vulnerable to the Bellcore attack and the attack objective consists of corrupting one of the two partial results, producing an invalid signature while remaining congruent to it modulo p or q. Three variants exist : Basic (unprotected), Shamir (coherence check between the two exponentiations in an extended domain p·r), and Aumüller (multiple integrity checks on parameters, output, and a cross domain check).

The two AES examples consist of the implementation of the full AES cipher and the isolated AES AddRoundKey step. The program's attack objective is to generate an invalid ciphertext.

The *Get Challenge collection* (GC) corresponds to the generation of a random challenge in an authentication protocol. The program's attack objective is to set the flag to `true` without being detected by the sentinel value in the buffer. The hardened variant stacks four software countermeasures - virtual call stack, doubled arguments, step counter, and loop counter.

7 result

In this section, we first present the results obtained for the `memcmps` code.
Next, we show the results for the `verify_pin` code. Finally, we discuss the findings related to the control-flow graph generated during the analysis.

7.1 Verify pin

This section presents the results for the `verifypin` test.
Table 1 : present the attacks identified by Lazart for the code C and Rust. The column "Version" indicates the version of the program and the implementation language. The second to fifth columns correspond respectively to the number of attacks reported by Lazart for 1 fault to 4 faults. Similarly, the next four columns present the number of *non-redundant* attacks for 1 fault to 4 faults. The last column "Countermeasures" enumerates the protection used for each version (the protection are the same for version X, and it's Rust counterpart) :

— *HB* : hardened boolean
— *FTL* : fixed time loops
— *INL* : inlined function

```rust
#[no_mangle]
#[inline(never)]
pub fn memcmps(a: &[u8; common::SIZE], b: &[u8; common::SIZE], len: i32) -> i32 {
    let mut result = common::FALSE_VAL;

    // `len` is faulted at EACH memcmp_stub call (independent IPs), mirroring the C
    // version where the parameter is reloaded at every use.
    if !memcmp_stub(a, b, mut_dl_sym!(len, "ip-len-1", common::ORDER) as u64) {
        result = mut_dl_sym!(result, "result-1", common::ORDER) ^ common::FALSE_VAL ^ common::MASK; // result = MASK
                                                                                                    //_LZ__opt_barrier!(); // opaque call: pins CFG,
                                                                                                    avoids IP duplication/merging
        if !memcmp_stub(a, b, mut_dl_sym!(len, "ip-len-2", common::ORDER) as u64) {
            result = mut_dl_sym!(result, "result-2", common::ORDER) ^ common::MASK2; // result = MASK ^ MASK2
                                                                                     //_LZ__opt_barrier!();
            if !memcmp_stub(a, b, mut_dl_sym!(len, "ip-len-3", common::ORDER) as u64) {
                result = mut_dl_sym!(result, "result-3", common::ORDER)
                    ^ common::TRUE_VAL
                    ^ common::MASK; // result = MASK2 ^ TRUE_VAL
                                    //_LZ__opt_barrier!();
                if !memcmp_stub(a, b, mut_dl_sym!(len, "ip-len-4", common::ORDER) as u64) {
                    result = mut_dl_sym!(result, "result-4", common::ORDER) ^ common::MASK2;
                    // result = TRUE_VAL
                    //_LZ__opt_barrier!();
                }
            }
        }
    }
    // DL fault on the returned value (mirror of the C `return result` IP).
    mut_dl_sym!(result, "result-return", common::ORDER)
}
```

FIGURE 5 – Secured version of memcmp in Rust

— *DPTC* : try counter decremented first
— *PTCBK* : try counter backup
— *LC* : loop counter verification
— *DC* : double call
— *DT* : double test
— *SC* : step counter

In the VP0 example, we observe 8 missing attacks when we compare to the C result. Since VP0 doesn't have hardened boolean, so it uses regular boolean, `rustc` reuse the branch condition in the `verify_pin` function, through XOR operations, to compute its return value, reducing the number of observable injection points on that path. All the higher-order attacks are redundant with those 8 one fault attacks.

The *redundancy analysis* regroups attacks that are variations of a simpler attack adding additional faults.

In the VP6 example, the non-redundant attacks count differs in 4-fault (while being the same without redundancy analysis) because `rustc` places some injection points earlier in the control flow. The IPs are therefore identified as the non-redundant ones, which changes the higher-order results.

7.2 memcmps

This section presents the results for the `memcmps` test.

Table 2 indicates for each version and implementation language (first column "Version") the results obtained for 1 to 4 faults - as for Table 1, the column "Attacks" shows attacks count while the "Non-redundant attacks" column shows the subset of non-redundant attacks.

Table 2 presents the attack found by Lazart for the code C and Rust.
Version 1 (V1) uses one mask and one call to `memcmp`.
Version 2 (V2) uses one mask and three calls to `memcmp`.
Version 3 (V3) uses two masks and four calls to `memcmp`.
Version 3 is also the one shown in Figure 2.

As shown in Table 2, the Rust implementation behaves exactly as the C implementation, showing the same result for 1 to 4 faults with a combination of TI and DL fault models.

7.3 Control Flow Graph

We also observed some major differences in how Rust compiles compared to C.
The intermediate code generated by LLVM revealed that Rust optimises certain conditional blocks differently than C.

However, we noticed differences in the control-flow graphs generated by the two language versions of the code presented in this study.
Specifically, Rust transforms variables with two possible values—depending on the evaluation of a condition, into a `select` instruction rather than a branch instruction. Similarly, Rust may use a `switch` instruction when a variable is compared to multiple values.

8 Conclusion

The first point of the report was to port the Rust language in Lazart. Lazart is now capable of analyzing Rust programs and being able to fault `select` and `switch` instructions.

On the second point of the article, there is now the possibility to generate a control flow graph of the code generated. The code generated could be the

Version	Attacks				Non-redundant attacks				Countermeasures
	1F	2F	3F	4F	1F	2F	3F	4F	
VP0	12	21	18	7	12	0	0	0	$\emptyset$
VP0 - rust	8	20	20	10	8	0	0	0	$\emptyset$
VP1	12	21	18	7	12	0	0	0	HB
VP1 - rust	12	21	18	7	12	0	0	0	HB
VP2	34	122	204	183	34	4	0	0	HB+FTL
VP2 - rust	34	122	204	183	34	4	0	0	HB+FTL
VP3	35	122	204	183	35	4	0	0	HB+FTL+INL
VP3 - rust	35	122	204	183	35	4	0	0	HB+FTL+INL
VP4	34	118	180	147	34	0	4	0	FTL+INL+DPTC+PTCBK+LC
VP4 - rust	34	118	180	147	34	0	4	0	FTL+INL+DPTC+PTCBK+LC
VP5	0	80	644	2566	0	80	248	184	HB+FTL+DPTC+DC
VP5 - rust	0	80	644	2566	0	80	248	184	HB+FTL+DPTC+DC
VP6	0	3	9	20	0	3	9	14	HB+FTL+INL+DPTC+DT
VP6 - rust	0	3	9	20	0	3	9	20	HB+FTL+INL+DPTC+DT
VP7	1	2	8	13	1	2	8	13	HB+FTL+INL+DPTC+DT+SC
VP7 - rust	1	2	8	13	1	2	8	13	HB+FTL+INL+DPTC+DT+SC

TABLE 1 – Comparison of attacks between `verify_pin` implementations

Version	Attacks				Non-redundant attacks			
	1F	2F	3F	4F	1F	2F	3F	4F
MCMPS1	16	32	28	8	16	0	0	0
MCMPS1 - rust	16	32	28	8	16	0	0	0
MCMPS2	4	32	404	1400	4	32	322	194
MCMPS2 - rust	4	32	404	1400	4	32	322	194
MCMPS3	4	32	216	2262	4	32	134	1468
MCMPS3 - rust	4	32	216	2262	4	32	134	1468

TABLE 2 – Comparison of attacks between `memcmps` implementations

mutated one or from the compiled program. The use of cfg shows that `rustc` uses `select` and `switch` rather than normal conditional block architecture.

And finally, in the last section, show that we almost get the same attack fault between both languages with `verify_pin` and `memcmps`. The minor differences are caused by Rust using the result of the condition with the help of XOR to return the value in `verify_pin` 0 and the loss of some attacks in version 6 of `verify_pin`, due to the attack not being in the same places as in C. However, it shows that the rust ported in Lazart is reliable and can be trusted. Furthermore, an idiomatic version of FISSC has been added to be closer to what a real programmer in Rust will have done.

Even if a successful version of Lazart is working with Rust, there are still elements to improve and issues to be fixed, one of them would be the panic function that causes Klee to fall into an infinite loop or the use of intrinsic optimization instructions (optimizations done at runtime) done by the LLVM.

Disclaimer on the Report

This report builds upon prior work, including :

1. An earlier report written during my **M1 MOSIG** internship.
2. The upcoming article **FDTC** ([20]), which is currently **under submission** for peer review.

While this report primarily highlights my personal contributions to the project, it is important to acknowledge that the broader research effort involves multiple collaborators. In particular, the development of Lazart's core architecture and the Rust port were made possible through the collective work of : - Laurent Mounier and Etienne Boespflug, Internship tutor and main research on Lazart tool. - Tommy Pradt, another research student, who focused on the Rust port of the core of Lazart and the development of idiomatic Rust attack surface examples (contribution present on **FDTC**).

—

Acknowledgments

I would like to express my sincere gratitude to **Tommy Pradt** for his essential contributions to the **core architecture of Lazart**, which were instrumental in enabling the Rust port and the development of idiomatic Rust attack surface examples involving the borrow checker.

—

9 Reference

[1] Nicholas D. Matsakis et Felix S. Klock. « The rust language ». In : *Ada Lett.* 34.3 (oct. 2014), p. 103-104. issn : 1094-3641. doi : 10.1145/2692956.2663188. url : https://doi.org/10.1145/2692956.2663188.

[2] Marie-Laure Potet et al. *Lazart : A Symbolic Approach for Evaluation the Robustness of Secured Codes against Control Flow Injections [SDTA14 version]*. Déc. 2015.

[3] Z. Segall et al. « FIAT - Fault injection based automated testing environment ». In : *Twenty-Fifth International Symposium on Fault-Tolerant Computing, 1995, ' Highlights from Twenty-Five Years'.* 1995, p. 394-. doi : 10.1109/FTCSH.1995.532663.

[4] Johannes Obermaier, Robert Specht et Georg Sigl. « Fuzzy-glitch : A practical ring oscillator based clock glitch attack ». In : *2017 International Conference on Applied Electronics (AE)*. IEEE. 2017, p. 1-6.

[5] Thomas Korak et Michael Hoefler. « On the effects of clock and power supply tampering on two microcontroller platforms ». In : *2014 Workshop on Fault Diagnosis and Tolerance in Cryptography*. IEEE. 2014, p. 8-17.

[6] Loic Zussa et al. « Investigation of timing constraints violation as a fault injection means ». In : *27th Conference on Design of Circuits and Integrated Systems (DCIS), Avignon, France*. Citeseer. 2012, p. 1-6.

[7] Loic Zussa et al. « Analysis of the fault injection mechanism related to negative and positive power supply glitches using an on-chip voltmeter ». In : *2014 IEEE International Symposium on Hardware-Oriented Security and Trust (HOST)*. IEEE. 2014, p. 130-135.

[8] Nidhal Selmane et al. « Security evaluation of application-specific integrated circuits and field programmable gate arrays against setup time violation attacks ». In : *IET information security* 5.4 (2011), p. 181-190.

[9] Bodo Selmke et al. « Precise laser fault injections into 90 nm and 45 nm sram-cells ». In : *Smart Card Research and Advanced Applications : 14th International Conference, CARDIS 2015, Bochum, Germany, November 4-6, 2015. Revised Selected Papers 14*. Springer. 2016, p. 193-205.

[10] Jean-Max Dutertre et al. « Laser fault injection at the CMOS 28 nm technology node : an analysis of the fault model ». In : *2018 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC)*. IEEE. 2018, p. 1-6.

[11] Amine Dehbaoui et al. « Injection of transient faults using electromagnetic pulses Practical results on a cryptographic system ». In : *ACR Cryptology ePrint Archive (2012)* (2012).

[12] Amine Dehbaoui et al. « Electromagnetic transient faults injection on a hardware and a software implementations of AES ». In : *2012 Workshop on Fault Diagnosis and Tolerance in Cryptography*. IEEE. 2012, p. 7-15.

[13] Yoongu Kim et al. « Flipping bits in memory without accessing them : An experimental study of DRAM disturbance errors ». In : *ACM SIGARCH Computer Architecture News* 42.3 (2014), p. 361-372.

[14] Louis Dureuil et al. « FISSC : a fault injection and simulation secure collection ». In : *Computer Safety, Reliability and Security*. T. 9922. Lecture Notes in Computer Science. Trondheim, Norway : Springer, sept. 2016, p. 3-11. doi : 10.1007/978-3-319-45477-1_1. url : https://hal.science/hal-03784107.

[15] Christopher Simon Liu et al. *SoK : A Beginner-Friendly Introduction to Fault Injection Attacks*. 2025. arXiv : 2509.18341 [cs.CR]. url : https://arxiv.org/abs/2509.18341.

[16] Jan Richter-Brockmann et al. « FIVER – Robust Verification of Countermeasures against Fault Injections ». In : *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2021.4 (août 2021). Artifact available at https://artifacts.iacr.org/tches/2021/a16, p. 447-473. doi : 10.46586/tches.v2021.i4.447-473.

[17] J. Arlat et al. « Fault injection for dependability validation : a methodology and some applications ». In : *IEEE Transactions on Software Engineering* 16.2 (1990), p. 166-182. doi : 10.1109/32.44380.

[18] Victor Arribas et al. « Cryptographic Fault Diagnosis using VerFI ». In : *2020 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. 2020, p. 229-240. doi : 10.1109/HOST45689.2020.9300264.

[19] William Pensec, Vianney Lapôtre et Guy Gogniat. *Automating Fault Injection through CABA Simulation for Vulnerability Assessment*. Colloque National du GDR SoC2. Poster. GDR SoC2, juin 2024. url : https://hal.science/hal-04727380.

[20] Anonymized. « FDTC : Study of Rust's attack surface against multiple fault injection ». In : (2026). Under submission.

[21] Lionel Rivière et al. « Combining High-Level and Low-Level Approaches to Evaluate Software Implementations Robustness Against Multiple Fault Injection Attacks ». In : *Foundations and Practice of Security - 7th International Symposium, FPS 2014, Montreal, QC, Canada, November 3-5, 2014. Revised Selected Papers*. 2014, p. 92-111.

[22] Minjae Kim, Eunsung Lee et Jaeyong Lee. « Hardening Rust's Result : Simple Source Code Transformations for Fault-Robust Bootloader ». In : *2026 IEEE International Conference on Consumer Electronics (ICCE)*. 2026, p. 1-5. doi : 10.1109/ICCE67443.2026.11449654.

[23] Jacopo Sini et al. « Improving Software Reliability with Rust : Implementation for Enhanced Control Flow Checking Methods ». In : *2025 Design, Automation & Test in Europe Conference (DATE)*. 2025, p. 1-7. doi : 10.23919/DATE64628.2025.10992995.

[24] Sébastien Michelland. « Compilation pour la sécurité matérielle : au-delà de la sémantique ». s343016. Thèse de doct. 2025. url : /document.

[25] Cristian Cadar, Daniel Dunbar et Dawson R. Engler. « KLEE : Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs ». In : *USENIX Symposium on Operating Systems Design and Implementation*. 2008. url : https://api.semanticscholar.org/CorpusID:2520229.

[26] *KLEE — klee-se.org*. https://klee-se.org/. [Accessed 01-06-2026].

Apendix A

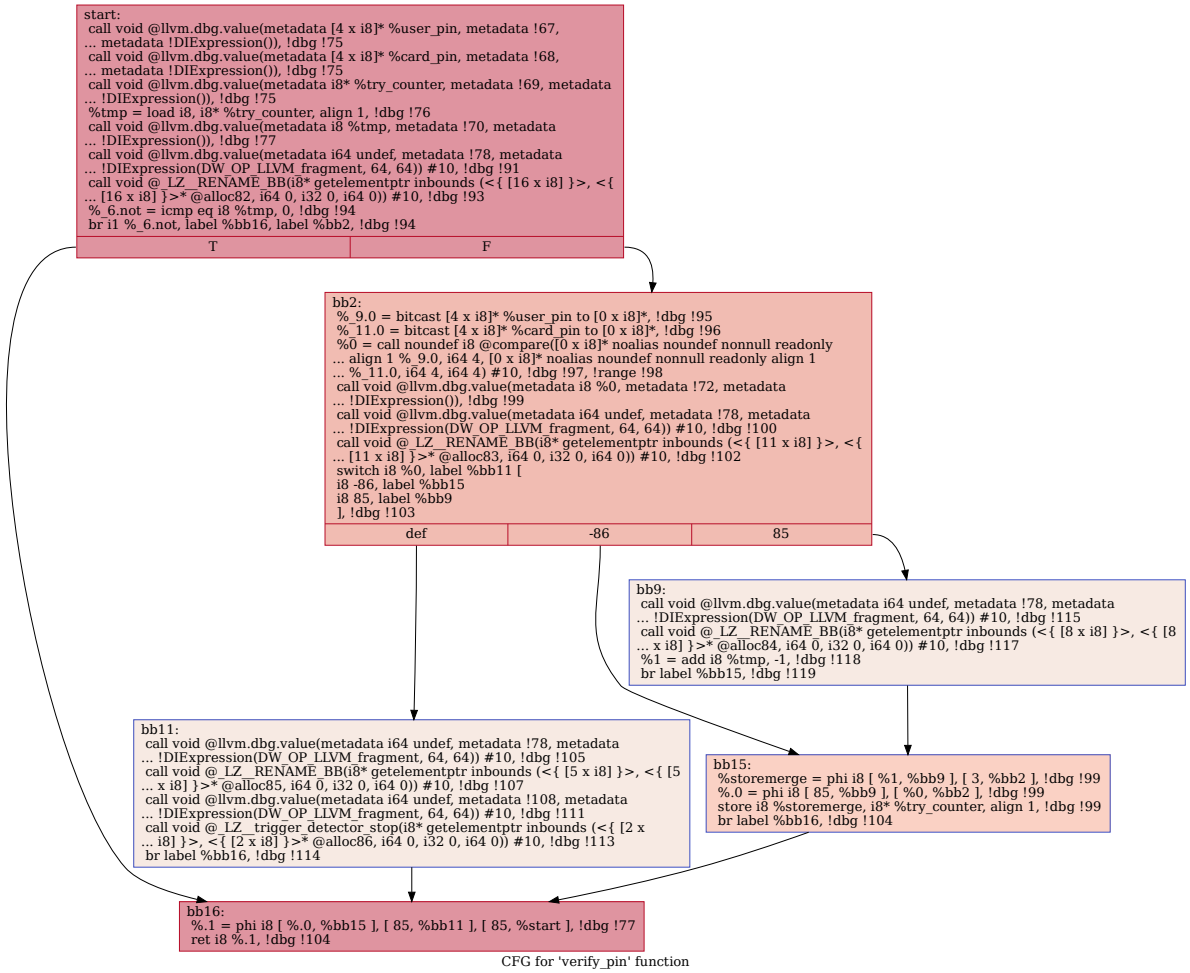

FIGURE 6 – verify_pin control flow graph